60 dakikalık oyun planı

Dakika	Adım	Ne yap / ne söyle
0-5	Tanışma	Kısa tanıtım. Format soruları: süre, tasarım/kod oranı, AI asistan izni, hangi dil?
5-12	**Gereksinimler**	Kullanıcılar kim? Ana 3-4 use case. **Fonksiyonel** + **fonksiyonel olmayan** (latency, availability, tutarlılık, güvenlik, multi-tenant, data residency). Kapsam dışını söyle. Kısa back-of-envelope.
12-25	**API sözleşmesi**	Kaynaklar (resources) → endpoint tablosu → 1-2 tam örnek request/response → hata modeli → auth → sayfalama → idempotency → uzun süren işler (202 + status). **Bu turun adı "API Development": burayı aceleye getirme.**
25-32	Veri modeli	Postgres tabloları, PK/unique, index'ler, tenant_id her yerde. Neyi Redis'te tutarsın?
32-45	**Mimari**	Whiteboard: Client → LB → Gateway → stateless API (GKE) → Postgres/Redis → kuyruk (Pub/Sub) → worker'lar. Read path ve write path'i ayrı anlat.
45-55	Derin dalış	Mülakatçının seçtiği konu. Hazır ol: ölçek, hata senaryoları, hot tenant, deploy/rollback, gözlemlenebilirlik, SLO, güvenlik.
55-60	Kapanış + sorular	"If I had more time I'd…" + 2 soru.

Kullanmaya hazır İngilizce cümleler

Başlarken

🗣️ *Before I jump into the design, I'd like to clarify the scope and a few requirements.*

🗣️ *Who are the consumers of this API: our own frontend, third-party integrations, or other internal services?*

🗣️ *I'll assume multi-tenant SaaS, EU data residency, and roughly X tenants. Does that sound right?*

🗣️ *Out of scope for now: billing and the admin UI. I'll mention them at the end if time allows.*

API aşaması

🗣️ *Let me start with the resources and the contract, because the contract is the hardest thing to change later.*

🗣️ *This operation can take minutes, so instead of holding the request open I'll return 202 with an operation resource the client can poll, plus an optional webhook.*

🗣️ *Clients will retry, so every mutating call accepts an Idempotency-Key.*

Mimari / trade-off

🗣️ *I'll keep the API layer stateless so we can scale it horizontally with an HPA, and push the heavy work to async workers.*

🗣️ *The trade-off here is eventual consistency: the user sees the change within a few seconds instead of immediately. For this use case that's acceptable, but not for permissions.*

🗣️ *I'd start with Postgres because it's what the team already runs well; I'd only move to something else when we hit a concrete limit.*

Takılınca / geri bildirim

🗣️ *That's a good point. Let me rethink that part.*

🗣️ *I see two options here. Let me compare them quickly and then pick one.*

🗣️ *I'm not 100% sure about the exact limit, but the way I'd find out is…*

Bitirirken

🗣️ *To summarize: the contract, the data model, the async pipeline, and how we'd operate it. The biggest risk I see is X, and I'd mitigate it with Y.*

Puan kaybettiren 8 hata

1. Gereksinim sormadan kutu çizmeye başlamak.
2. API'yi atlayıp direkt mimariye geçmek (bu turun adı API!).
3. Hata modelini, sayfalamayı, idempotency'yi unutmak.
4. "Kafka/Cassandra koyalım" deyip **neden** sorusuna cevap verememek. AI Company'nin stack'i: **GCP, Kubernetes, Terraform, Postgres, Redis, TypeScript.** Önce bunları kullan.
5. **Multi-tenancy'yi (tenant izolasyonu)** unutmak. Kurumsal SaaS'ta en kritik konu.
6. Operasyon tarafını unutmak: deploy, rollback, SLO, alert, on-call. **SRE/Infra** rolünde en güçlü yanın bu olmalı.
7. Mülakatçının ipucunu savunmaya geçmek. Şirketlerin mülakat rehberleri açıkça diyor: **geri bildirime açık ol, adapte ol.**

Önceki pratik tasarımlar işe yarar mı?

Kısa cevap: Evet, ama olduğu gibi değil. Önceki pratik tasarımlar (Spam Lookup, Contact Sync, kurumsal IAM) içindeki **kalıplar** birebir işe yarar: gateway + rate limit, idempotency key, async kuyruk + worker, "single writer per key", cache katmanları, SLO'lar, data residency, JWT/JWKS. **Değişmesi gereken üç şey:** (1) bu turda **API sözleşmesi** merkezde olmalı, kutular ikinci planda; (2) AI Company'nin stack'ine çevir (GCP + Postgres + Redis + TS, Cassandra değil); (3) her şeyi **multi-tenant enterprise** gözüyle anlat (tenant izolasyonu, kurumsal SSO, izin farkındalığı).

Kalıp transfer tablosu

Pratik tasarımdaki kalıp	Nerede vardı	AI Company'de nerede kullanırsın
202 Accepted + kuyruk + worker	Contact Sync, Spam Report	Knowledge sync, doküman işleme, embedding, toplu import. AI Company'de LLM/tool çağrıları yavaş olduğu için **neredeyse her şey async**.
Idempotency key (Redis SET NX, TTL)	Contact Sync	Her POST'ta `Idempotency-Key`. Ayrıca DB'de `UNIQUE(tenant_id, idempotency_key)` → Redis düşse bile güvenli.
Partition key ile "single writer"	Kafka key=user_id / number	Pub/Sub **ordering key** = document_id veya task_id. Aynı nesneye iki worker yarışmaz.
"Sadece daha yeniyse yaz"	Cache Updater	`ON CONFLICT … DO UPDATE … WHERE excluded.version > t.version`: sırasız mesajlarda eski veri yeniyi ezmez.
Değişmeyen veri = 0 yazma	Contact Sync (hash diff)	Doküman `content_hash` aynıysa yeniden chunk/embedding yok → **LLM maliyeti** düşer. Bir AI şirketi için çok güçlü bir cümle.
Hot key / hot partition	Hot number, scammer	**Hot tenant / noisy neighbor**: büyük müşteri 5M doküman onboard ederken diğerleri beklememeli → tenant başına adil kuyruk, eşzamanlılık limiti.
Rate limit: fail-open vs fail-closed	Spam Lookup	Okuma/arama: fail-open. LLM token bütçesi, auth, yazma: fail-closed. Lokal (pod içi) limit + global (Redis) limit.
L1 in-process + L2 Redis + singleflight	Spam Lookup	Tenant config, kullanıcı grup üyelikleri (ACL), feature flag'ler, JWKS.
Backpressure: full resync'i önce kes	Contact Sync	Yük altında önce **ilk senkronizasyonları** yavaşlat, artımlı değişiklikleri ve **izin iptallerini** asla yavaşlatma.
Data residency (home region)	Contact Sync	Kurumsal müşteri: EU verisi EU'da. Tenant → region eşlemesi, bölge başına ayrı stack (cell).
OIDC, JWT, lokal JWKS doğrulama	Kurumsal IAM	Kurumsal SSO (Entra/Okta), servisler arası JWT, **tool çağrıları için kullanıcı adına OAuth token**.
Strangler migration	Kurumsal IAM	API versiyon geçişleri, eski sistemden yeni sisteme geçiş, ana şirket sistemleriyle entegrasyon.
SLO + operasyonel metrikler	Hepsi	Aynen kullan. Buna ek olarak LLM'e özgü metrikler: TTFT, tokens/s, tenant başına maliyet.

Stack çeviri tablosu (söylerken buna göre konuş)

Pratik tasarımda	AI Company / GCP karşılığı	Not
Cassandra	**Cloud SQL / AlloyDB Postgres** (+ partitioning, read replica)	Cassandra'yı sadece gerçek bir yazma sınırına ulaşınca öner. Postgres'te `tenant_id` ile hash partition, RLS.
Kafka	**Pub/Sub** (ordering key, DLQ, retry policy) veya Redis Streams	Kafka da olur ama "managed + ekibin bildiği" argümanı güçlü. Replay gerekiyorsa Kafka/GCS arşivi.
S3 + presigned URL	**GCS signed URL** (resumable upload)	Büyük dosya API'yi bloklamasın.
Redis	**Memorystore Redis**	Rate limit, idempotency, cache, event stream.
Istio/Envoy, CDN + WAF	**Cloud Load Balancing + Cloud Armor**, GKE Gateway/Envoy	WAF, TLS, DDoS, tenant bazlı rate limit.
KEDA, HPA	Aynı (GKE)	Pub/Sub backlog'a göre KEDA autoscaling.
Vault	**Secret Manager** + Workload Identity	Pod'da statik key yok.
ArgoCD, Prometheus, Grafana, OTel, Loki	Aynı ya da Cloud Monitoring/Logging/Trace	Kendi deneyimin, rahat konuş.

API tasarımı kopya kağıdı

1. Kaynak odaklı URL'ler

```
GET    /v1/sources?cursor=..&limit=50   list
POST   /v1/sources                        create → 201 + Location
GET    /v1/sources/{id}                   read
PATCH  /v1/sources/{id}                   partial update
DELETE /v1/sources/{id}                   → 204 (veya 202 async)
POST   /v1/sources/{id}/syncs             alt kaynak (sub-resource)
POST   /v1/tasks/{id}:cancel
aksiyon (Google AIP stili)
```

İsimler çoğul, fiil yok. Aksiyon şart ise `:cancel` veya `/cancel`. Tenant'ı URL'e koyma, **token'dan** çıkar (spoofing riski yok).

2. Status kodları

200	OK (GET/PATCH)	400	Bozuk istek / validasyon
201	Created + Location	401	Kimlik yok / geçersiz
202	Kabul edildi, async	403	Yetki yok
204	İçerik yok	404	Yok (veya başka tenant'ın! 403 yerine 404 → varlık sızdırma)
304	Not Modified (ETag)	409	Çakışma / farklı payload ile aynı idempotency key
		412	If-Match başarısız (eşzamanlı güncelleme)
		422	Semantik hata
		429	Rate limit + Retry-After
		500/503	Sunucu / geçici; 503 + Retry-After

3. Hata formatı (RFC 9457 problem+json)

```
{ "type": "https://api.example.com/errors/quota-exceeded",
  "title": "Token quota exceeded",
  "status": 429,
  "detail": "Tenant t_42 used 100% of daily tokens",
  "code": "QUOTA_EXCEEDED",        // makine için sabit
  "request_id": "req_9f1c",        // log/trace ile eşleşir
  "errors": [{"field":"limit","msg":"max 100"}] }
```

4. Sayfalama: cursor (offset değil)

```
GET /v1/documents?limit=50&cursor=eyJpZCI6...
→ { "data": [...], "next_cursor": "eyJ...",
"has_more": true }
SQL: WHERE (created_at, id) < ($1, $2)
     ORDER BY created_at DESC, id DESC LIMIT 51
```

Offset büyük tablolarda yavaş ve araya ekleme olunca kayar. Cursor = opak, base64 (son sıralama anahtarı). Limit'e üst sınır koy.

5. Idempotency

```
POST /v1/syncs
Idempotency-Key: 7c1e-...  (istemci üretir, UUID)
1) (tenant, key) ara → varsa: aynı hash ise kayıtlı
   cevabı dön; farklı payload ise 409
2) yoksa: INSERT ... (UNIQUE tenant_id, key)
3) işi yap, cevabı kaydet (TTL 24 saat)
```

PUT/DELETE zaten idempotent. Ağ hatasında istemci **aynı key ile** exponential backoff + jitter ile tekrar dener.

6. Uzun süren işler (LRO)

```
POST /v1/sources/c1/syncs → 202
Location: /v1/syncs/s_77
{ "id":"s_77", "status":"queued" }
GET /v1/syncs/s_77 → {status:"running", progress:
{done:1200,total:5000}}
Durumlar: queued → running → succeeded | failed |
cancelled
```

Polling + opsiyonel **webhook** (HMAC imzalı) veya **SSE** stream. LLM çıktısı için SSE.

7. Eşzamanlı güncelleme

`ETag: "v12"` → istemci `If-Match: "v12"` ile PATCH. Uyuşmazsa 412. DB'de `UPDATE … WHERE id=$1 AND version=$2` (optimistic locking).

8. Auth

- Kullanıcı: OIDC (Auth Code + PKCE) → kısa ömürlü JWT access token. Servis lokal JWKS ile doğrular (kurumsal IAM'de olduğu gibi).
- Makine/entegrasyon: OAuth2 client credentials veya API key (DB'de **hash**'i, prefix ile tanınır: `sk_live_ab12…`), **scope**'lar.
- Authz: token'da kaba roller; ince izin servis içinde. Her sorguda `tenant_id` filtresi + Postgres **RLS**.

9. Rate limit başlıkları

```
429 Too Many Requests
Retry-After: 12
RateLimit-Limit: 600   RateLimit-Remaining: 0
RateLimit-Reset: 12
```

Token bucket (Redis + Lua, atomik). Boyutlar: tenant, kullanıcı, API key, endpoint. LLM için **token bazlı** bütçe.

10. Versiyonlama ve evrim

- URL'de major: `/v1`. Ek alan eklemek **kırmaz**; alan silmek, anlam değiştirmek, zorunlu alan eklemek kırar.
- İstemciler bilinmeyen alanları yok saymalı (tolerant reader). Enum'a değer eklemek dikkat ister.
- Deprecation: `Deprecation` + `Sunset` header, kullanım metrikleri, iletişim.
- Sözleşme önce: **OpenAPI** spec → codegen, contract test, lint (Spectral) CI'da. (DX cümlesi!)

11. Diğer kontrol listesi

Request ID / trace header (`traceparent`) · max body size · zaman ISO-8601 UTC · para integer · ID'ler opak string, prefix'li (`task_`, `doc_`) · filtre/sıralama parametreleri · `fields` / `expand` · gzip · CORS · audit log.

Solutions design ve infra kopya kağıdı (AI Company stack'i)

GCP yapı taşları

GKE (regional)	Node pool'lar ayrı: api, workers, spot/preemptible (batch embedding). Workload Identity.
Cloud SQL / AlloyDB	Postgres HA (zonal failover), read replica, PITR backup. PgBouncer.
Memorystore	Redis: cache, rate limit, idempotency, stream.
Pub/Sub	At-least-once, ordering key, ack deadline, DLQ topic, retry policy.
Cloud Tasks / Scheduler	Zamanlanmış/gecikmeli işler, hız sınırlı dispatch.
GCS	Ham dosya, signed URL, lifecycle kuralları.
Cloud LB + Cloud Armor	Global L7, TLS, WAF, DDoS, IP allowlist (enterprise).
Secret Manager / KMS	OAuth token'ları şifreli (envelope encryption), CMEK müşteri anahtarı.

Kubernetes: söylenmesi gerekenler

- **Probes:** readiness (trafik al), liveness (yeniden başlat; DB'yi kontrol etme!), startup.
- **HPA** CPU/RPS'e göre; **KEDA** kuyruk backlog'una göre. requests/limits, **PDB**, topology spread (zone).
- **Graceful shutdown:** SIGTERM → readiness false → preStop sleep → in-flight bitir → `terminationGracePeriodSeconds`. SSE/uzun bağlantılar için kritik.
- **Rollout:** rolling + maxUnavailable 0; riskli değişiklikte **canary** (Argo Rollouts, metrik bazlı otomatik rollback). Feature flag ile deploy ≠ release.
- NetworkPolicy, namespace per servis/takım, ResourceQuota.

Terraform / IaC

- Modüller (gke-cluster, cloudsql, pubsub-topic+dlq), ortam başına state (GCS backend + lock).
- PR'da `terraform plan` yorumu, apply sadece CI'dan (Atlantis / GH Actions + OIDC → Workload Identity Federation, statik key yok).
- Drift tespiti, policy-as-code (OPA/Conftest). App config: Helm + ArgoCD (GitOps).

Postgres: production ipuçları

- Connection pool (PgBouncer, transaction mode); pod sayısı × pool < max_connections.
- `EXPLAIN (ANALYZE, BUFFERS)`, `pg_stat_statements`, eksik index, N+1.
- Büyük tablolar: partitioning (zaman / tenant hash), autovacuum ayarı, bloat.
- Online migration: expand → backfill (batch) → contract. `CREATE INDEX CONCURRENTLY`. Kilitlenen ALTER'a dikkat, `lock_timeout`.
- İş kuyruğu: `SELECT … FOR UPDATE SKIP LOCKED` (outbox / job table).
- **Transactional outbox:** DB yazımı + event aynı transaction'da → relay Pub/Sub'a basar. "DB'ye yazdım ama event kayboldu" problemi yok.
- Multi-tenant: `tenant_id` her tabloda, composite index'te ilk kolon, RLS.

Redis kalıpları

- Cache-aside + TTL jitter + singleflight (stampede). Negative cache.
- Rate limit: token bucket Lua script (atomik). Idempotency: `SET key val NX EX 86400`.
- Distributed lock: kısa TTL + fencing token (Redlock'a güvenme, kritik iş DB'de).
- Streams: `XADD` / `XREAD BLOCK` → canlı event fan-out, consumer group.
- Eviction policy, memory alarmı, hot key, persistence kapalı mı açık mı?

Güvenilirlik araç kutusu

- **Timeout** her dış çağrıda (deadline propagation). **Retry** sadece idempotent işte, exp. backoff + jitter, retry budget.
- **Circuit breaker** (LLM sağlayıcı, kaynak API'ler), **bulkhead** (tenant/bağımlılık başına havuz).
- **Load shedding**: önemsiz işi düşür, önemliyi koru. Kuyrukta **öncelik şeritleri**.
- DLQ + alarm + replay aracı. Poison message izolasyonu.
- Degrade: LLM yoksa arama sonuçları yine dönsün; fallback model.

SRE dili

- **SLI** (ölçülen) → **SLO** (hedef, ör. 99.9% / 30 gün = ~43 dk hata bütçesi) → **error budget** politikası (bütçe bitti → feature freeze, güvenilirlik işi).
- Alarm **semptoma** göre (burn rate, multi-window: 1h & 6h), sebebe göre değil. Her alarmın runbook'u var.
- RED (Rate, Errors, Duration) servisler için; USE (Utilization, Saturation, Errors) kaynaklar için.
- Incident: tespit → IC atanır → mitigate (rollback önce!) → iletişim → blameless postmortem → action item.
- OTel trace: `tenant_id`, `task_id` span attribute. Log'da PII yok.

Back-of-envelope ezber sayıları

1 gün	86.400 sn ≈ 10^5
1M istek/gün	≈ 12 rps ort., tepe ×3-5
Little's law	eşzamanlı = rps × süre (120 rps × 20 sn = 2.400 açık stream)
Redis	~100k ops/sn/node, <1 ms
Postgres (iyi index)	~5-20k basit sorgu/sn/primary; tek satır okuma ~1 ms
Aynı bölge ağ RTT	~0,5 ms; EU↔US ~80-100 ms
LLM çağrısı	TTFT 0,3-2 sn; tam cevap 2-30 sn → **her zaman async/stream**
Embedding vektörü	768 float32 ≈ 3 KB; 1536 ≈ 6 KB
1 sayfa metin	~500 token ≈ 2 KB
99.9% / 99.99%	ayda ~43 dk / ~4,3 dk kesinti

Gereksinim soruları bankası

Kim kullanıyor (UI / 3. parti / iç servis)? Kaç tenant, en büyüğü ne kadar? Okuma:yazma oranı? Latency hedefi? Tutarlılık: hemen mi, saniyeler mi? Veri hassas mı (PII, residency)? Silme/GDPR? Multi-region? Mevcut sistemlerle entegrasyon? Maliyet hassasiyeti (LLM token!)?

Senaryo A · Knowledge Sync Platform

Contact Sync'in kardeşi: yazma ağırlıklı, async pipeline. Agent ürününün kalbi: "access all your company's apps, knowledge and data".

> **Mülakatçı:** 🗣️ "*Our agent product lets enterprises connect sources like Google Drive, Confluence, SharePoint and Slack, so agents can answer questions using company knowledge. Design the API and the backend that keeps these sources in sync and makes the content searchable. Focus on the API and how you would run it in production.*"

Sorulacak netleştirme soruları

1. API'yi kim kullanıyor? (Admin UI, agent runtime, müşteri entegrasyonları?)
2. Kaç tenant, en büyük tenant kaç doküman? Değişim oranı?
3. Ne kadar taze olmalı? (Yeni doküman kaç dakikada aranabilir?)
4. **İzinler:** kullanıcı sadece kaynakta görebildiğini mi görmeli? (Evet → kritik.)
5. Silme / erişim iptali ne kadar hızlı yansımalı?
6. Kaynak API'lerin kota/limitleri? Webhook desteği var mı?
7. Data residency (EU tenant verisi EU'da)?
8. Embedding/LLM maliyeti bir kısıt mı?

Gereksinimler

Fonksiyonel: (1) Admin bir kaynağı bağlar (OAuth) ve kapsamı seçer. (2) İlk tam senkronizasyon + sonra artımlı. (3) Senkronizasyon durumu/hataları görünür. (4) Agent/kullanıcı **izin farkındalıklı** arama yapar. (5) Bağlantı silinince tüm veri temizlenir.

Fonksiyonel olmayan: Artımlı tazelik p95 < 10 dk · izin iptali p99 < 5 dk · arama p95 < 300 ms · API %99.9 · **sıfır tenant'lar arası sızıntı** · tenant'lar arası adalet · maliyet kontrolü.

Kapsam dışı: LLM cevap üretimi (Senaryo B), billing, source UI.

Basit v1 (önce bunu çiz, 5 dk)

Knowledge Sources API (Go, stateless) + Postgres (sources, sync_runs, documents, chunks + pgvector) + bir Pub/Sub topic + tek worker deployment. Webhook yok: her N dakikada delta token ile **poll**. Arama: Postgres'te ACL filtresi + vektör sorgusu.

🗣️ "*That's v1. The things that break first at scale are source quotas, one big tenant starving the others, and permission freshness. Which one should I go into?*"

Back-of-envelope

Tenant	500 kurumsal müşteri
Doküman	ort. 300k / tenant → **150M doküman**; en büyük tenant 5M
Değişim	günde %1 → 1,5M/gün ≈ **17/sn ort.**, mesai tepesi ×5 ≈ 90/sn
Onboarding dalgası	Büyük tenant: 5M doküman, 24 saatte bitsin → ~60 doc/sn **tek tenant** için
Chunk	ort. 6 chunk/doküman → 900M chunk
Vektör	900M × 3 KB (768 dim) ≈ **2,7 TB** + metin ~2 TB
Embedding	Artımlı: 1,5M × 6 = 9M chunk/gün (hash ile değişmeyenler atlanınca daha az)
Arama	200k DAU × 20 sorgu = 4M/gün ≈ 50 rps ort., tepe ~250 rps

Çıkarım: Darboğaz QPS değil; **kaynak API kotaları, embedding maliyeti ve tenant adaleti**. Bunu söylemek kıdem gösterir.

Kaynaklar (resources)

`Source` (tenant'ın bağlı kaynağı) · `SyncRun` (bir senkronizasyon işi, LRO) · `Document` (indekslenmiş öğe) · `SearchQuery` (aksiyon) · `Webhook` (kaynaktan gelen bildirim)

> **Açılış cümlesi:** 🗣️ "*The interesting problems here are not QPS. They're permission correctness, fairness between tenants, source API quotas, and embedding cost. I'll design the API first, then the pipeline around those four.*"

Senaryo A · API sözleşmesi ve veri modeli

Endpoint'ler

Endpoint	Cevap	Not
`POST /v1/sources`	201	`{type:"google_drive", auth:{oauth_code}, scope:{folders:[…]}}`. Token'ı Secret Manager'a/KMS ile şifreli DB'ye koy, cevapta asla dönme.
`GET /v1/sources?cursor=&limit=`	200	Liste, `status`: active | paused | auth_error | deleting.
`PATCH /v1/sources/{id}`	200 / 412	Kapsam değiştir, durdur. `If-Match` ile.
`DELETE /v1/sources/{id}`	**202**	Veri temizliği async; status=deleting. Arama hemen hariç tutar.
`POST /v1/sources/{id}/syncs`	**202** + Location	`Idempotency-Key`. `{mode:"incremental"|"full"}`. Zaten çalışan varsa: **mevcut run'ı 200 ile dön** (veya 409, seçimini söyle).
`GET /v1/syncs/{id}`	200	LRO durumu + ilerleme + hata özeti.
`POST /v1/syncs/{id}:cancel`	202	Kooperatif iptal: worker her sayfada kontrol eder.
`GET /v1/sources/{id}/documents?status=failed`	200	Hangi doküman neden başarısız? (Destek ve DX için.)
`POST /v1/hooks/{source}`	204	Kaynaktan gelen değişim bildirimi. **İmza doğrula**, hemen kuyruğa at, 1 sn içinde dön.
`POST /v1/search`	200	**Kullanıcı token'ı** ile. `{query, top_k, filters:{source_ids, updated_after}}` → sonuçlar + atıf (url, başlık, parça).

Örnek: sync başlat

```
POST /v1/sources/src_8f2/syncs
Authorization: Bearer eyJ…      (tenant = token'dan)
Idempotency-Key: 3b6c9d0e-…
{ "mode": "incremental" }

202 Accepted
Location: /v1/syncs/syn_51a
{ "id": "syn_51a", "source_id": "src_8f2",
  "mode": "incremental", "status": "queued",
  "created_at": "2026-11-06T09:12:03Z" }
```

Örnek: durum

```
GET /v1/syncs/syn_51a → 200
{ "id":"syn_51a", "status":"running",
  "progress": {"discovered":5200,"processed":3100,
               "skipped_unchanged":2400,"failed":7},
  "started_at":"…", "last_error": null }
```

Örnek: arama

```
POST /v1/search
{ "query": "parental leave policy Sweden", "top_k":
8 }
200
{ "results": [
   { "document_id":"doc_19c", "title":"Leave Policy
SE",
     "url":"https://drive.google.com/…",
"score":0.82,
     "snippet":"…480 days of parental leave…",
     "source":"google_drive", "updated_at":"…" } ],
  "request_id":"req_77" }
```

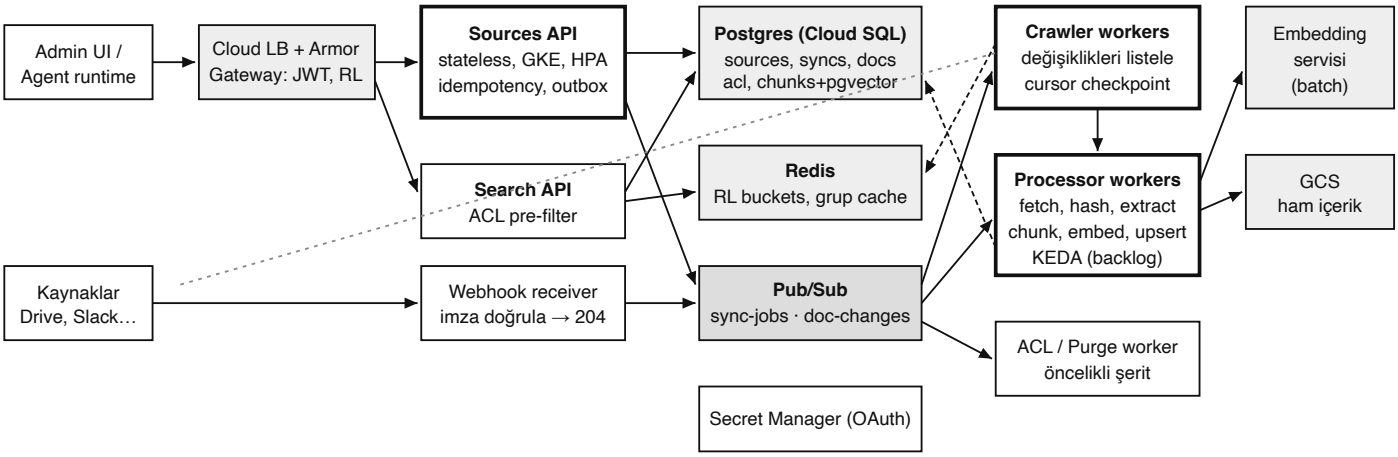
Durum makinesi (SyncRun): `queued → running → succeeded | failed | cancelled`. Geçişler tek yerde, DB'de `UPDATE … WHERE status='running'` ile korunur.

Veri modeli (Postgres, her tabloda tenant_id + RLS)

```
sources (tenant_id, id PK, type, status, scope jsonb, secret_ref, delta_cursor text,
           version int, created_at)
sync_runs  (tenant_id, id PK, source_id, mode, status, discovered, processed, failed,
            idempotency_key, started_at, finished_at,
            UNIQUE(tenant_id, idempotency_key),
            UNIQUE(source_id) WHERE status IN ('queued','running'))   -- 1 aktif run
documents  (tenant_id, id PK, source_id, external_id, title, url, content_hash,
            source_version bigint, acl_version bigint, deleted_at, updated_at,
            UNIQUE(source_id, external_id))
document_acl (tenant_id, document_id, principal_id)  -- user / group / "domain:acme.com"
chunks     (tenant_id, document_id, chunk_no, text, embedding vector(768), PK(document_id, chunk_no))
-- partition: HASH(tenant_id); büyük tenant → ayrı partition / ayrı vektör index'i
```

Neden partial unique index? Aynı kaynak için aynı anda iki sync çalışmasını DB garanti eder; uygulama yarışı (race) olsa bile. **Vektör:** başlangıçta **pgvector** (HNSW) — operasyonel tek sistem; büyük tenant'lar veya >birkaç yüz milyon vektörde ayrı vektör DB'ye (Vertex AI Vector Search vb.) geçiş planı. Trade-off'u söyle.

Senaryo A · Mimari ve derin dalışlar



Kesikli: DB yazımı / Redis rate limit. Gri: crawler kaynak API'yi çağırır (tenant+kaynak başına token bucket).

Akışlar

Sync: API → `sync_runs` INSERT + outbox (aynı txn) → `sync-jobs` → Crawler: kaynaktan sayfa sayfa değişiklik listesi (delta token), her sayfadan sonra **cursor checkpoint** (crash olursa kaldığı yerden) → her değişiklik `doc-changes` (ordering key = document_id) → Processor.

Processor: içeriği çek → `content_hash` aynı mı? **Aynıysa embedding yok**, sadece metadata/ACL güncelle → farklıysa chunk + batch embed → tek transaction'da eski chunk'ları sil + yenileri yaz + `documents` upsert `WHERE excluded.source_version > source_version`.

Webhook: imza doğrula, 204 dön, sadece "bu doküman değişti" bilgisini kuyruğa at (payload'a güvenme, kaynaktan tekrar çek).

İzin iptali / silme: ayrı **yüksek öncelikli** topic + worker. Re-embed yok, sadece `document_acl` / `deleted_at`. Arama hemen etkilenir.

Derin dalış 1 · İzin doğruluğu (en önemli)

- Arama: kullanıcının principal kümesi = user_id + grupları (IdP / kaynaktan, Redis'te 5 dk cache). Sorgu **önce** ACL ile filtreler, sonra vektör benzerliği (pre-filter). Post-filter → top_k boş kalabilir ve sızıntı riski.
- **Fail-closed:** ACL bilinmiyorsa dokümanı gösterme. Grup servisi çökerse: son bilinen cache, o da yoksa hata.
- Periyodik **reconciliation** job: kaynaktaki ACL ile karşılaştır (webhook'lar kaçabilir).
- Tenant izolasyonu: RLS + her sorguda tenant_id + entegrasyon testi "tenant A token'ı ile B'nin dokümanı 404".

Derin dalış 2 · Kaynak kotaları ve backoff

- Redis token bucket: `(tenant, source)` başına; kaynağın kotasına göre ayarlı. 429 → `Retry-After`'a uy, run'ı "throttled" işaretle, mesajı gecikmeli tekrar kuyruğa koy (worker'ı uyutma).
- Kaynak başına circuit breaker: Confluence çökerse Drive senkronu etkilenmez (bulkhead).
- OAuth token süresi dolarsa: refresh; refresh başarısızsa source `auth_error`, admin'e bildirim, run'lar durur.

Derin dalış 3 · Noisy neighbor (hot tenant)

- Spam Lookup tasarımındaki "hot number"ın karşılığı. Büyük tenant onboard olurken 5M doküman kuyruğu doldurur.
- Çözüm: iki şerit: **backfill** (ilk sync) ve **live** (artımlı + ACL). Tenant başına eşzamanlılık limiti (ör. max 20 worker slot). Fair scheduling: job tablosundan round-robin tenant seçimi (`FOR UPDATE SKIP LOCKED`).
- Embedding: backfill için spot node pool + batch API (daha ucuz), live için düşük gecikmeli yol.

Hata senaryoları

Worker crash	Pub/Sub ack gelmez → tekrar teslim. Upsert idempotent (deterministik PK).
Bozuk dosya (poison)	N deneme → DLQ, doküman `failed`, run devam eder.
Postgres failover	~30-60 sn yazma hatası → worker'lar retry, API 503 + Retry-After. Arama read replica'dan devam.
Embedding sağlayıcı çöktü	Circuit breaker, kuyruk birikir (tazelik SLO'su yanar, veri kaybı yok), alarm.
Kötü deploy	Canary + otomatik rollback; GCS'te ham içerik → yeniden işleme (replay).

SLO ve alarmlar

API availability 99.9% · artımlı tazelik p95 < 10 dk · ACL iptali p99 < 5 dk · arama p95 < 300 ms. **Metrikler:** backlog yaşı (en eski mesaj), tenant başına lag, DLQ oranı, 429 oranı (kaynak bazlı), embedding maliyeti/tenant, "skipped_unchanged" oranı.

Senaryo B · Agent Tasks API (streaming, tool calls)

Spam Lookup'ın kardeşi: gecikmeye duyarlı, okuma/stream ağırlıklı, rate limit kritik. Bu sefer pahalı kaynak Cassandra değil **LLM token'ı**.

> **Mülakatçı:** 🗣️ "*Users talk to our agents from the web app, Slack, and through a public API. An agent task can call tools like search, Salesforce or Jira and can take from a few seconds to several minutes. Users want to see the answer streaming. Design the API and the system behind it.*"

Netleştirme soruları

1. İstemciler: web (stream ister), Slack bot (mesaj güncelleme), 3. parti (webhook ister)?
2. Bir task en fazla ne kadar sürer? İnsan onayı (tool approval) var mı?
3. Kullanıcı bağlantıyı kaybedince task devam etmeli mi? (Evet.)
4. LLM sağlayıcıları: biri mi, birden fazla mı? Tenant başına kota/limit?
5. Tool'lar yazma yapıyor mu (Jira ticket aç)? → **tekrar edilmemeli**.
6. Konuşma geçmişi ne kadar saklanıyor? Residency?

Gereksinimler

Fonksiyonel: task başlat, canlı token/olay akışı, kopunca kaldığı yerden devam, iptal, durum sorgulama, geçmiş, async mod + webhook, tool onayı.

Fonksiyonel olmayan: ilk olay (TTFT, model hariç overhead) p95 < 500 ms · stream kopsa da **task kaybolmaz** · tool yan etkileri en fazla 1 kez · tenant başına token bütçesi · deploy'lar task'ları öldürmez · %99.9.

Back-of-envelope

Kullanıcı	200k DAU × 10 task/gün = **2M task/gün**
Hız	≈ 23 task/sn ort.; 09:00 CET tepe ×5 ≈ **120 task/sn**
Süre	ort. 20 sn → Little: 120 × 20 = **~2.400 eşzamanlı stream** (uzun kuyrukla ~10k planla)
LLM çağrısı	~3 çağrı/task × (8k giriş + 500 çıkış token) ≈ 25k token/task
Token/gün	2M × 25k = **50B token/gün** → maliyet ve sağlayıcı limiti asıl kısıt
Olaylar	~300 olay/task (token delta'ları) × 120/sn = 36k olay/sn tepe → Redis Streams rahat
Depolama	2M task × ~20 KB = 40 GB/gün (mesajlar + tool sonuçları)

Çıkarım: CPU değil; **uzun açık bağlantılar**, **LLM kotası** ve **dayanıklı yürütme** (crash/deploy sonrası devam) zor kısımlar.

Kaynaklar

`Conversation` · `Message` · `Task` (LRO, durum makinesi) · `TaskEvent` (sıralı, `seq`'li) · `ToolCall`

Task durum makinesi

```
queued → running ⇄ requires_action (tool onayı bekliyor)
running → completed | failed | cancelled | expired
(terminal durumlardan çıkış yok; geçişler tek fonksiyonda)
```

> **Ana fikir:** 🗣️ "*I want to separate the connection that streams to the client from the process that executes the task. Then a dropped connection or a deploy doesn't kill the task, and the client can resume from the last event it saw.*"

Basit v1 (önce bunu çiz, 5 dk)

Agent Tasks API + Postgres (tasks, messages) + kuyruk + agent worker'ları. İstemci önce **polling** yapar: `POST /tasks` → 202, `GET /tasks/{id}`. Sonra 2. adım: SSE + Redis stream ile canlı akış ve kaldığı yerden devam.

🗣️ "*I'd ship polling first because it's simple and robust, then add streaming once the task lifecycle is solid.*"

Senaryo B · API sözleşmesi

Endpoint	Cevap	Not
`POST /v1/conversations`	201	`{agent_id, metadata}`
`POST /v1/conversations/{id}/tasks`	200 SSE *veya* 202	`{input, stream:true|false, webhook_url?}` + `Idempotency-Key`. `Accept: text/event-stream` → stream, yoksa 202 + Location.
`GET /v1/tasks/{id}`	200	Durum, kullanım (token), sonuç mesajı.
`GET /v1/tasks/{id}/events`	200 SSE	**Devam etme:** `Last-Event-ID: 41` header'ı veya `?after=41` → 42'den itibaren.
`POST /v1/tasks/{id}:cancel`	202	İdempotent; terminal ise 200 mevcut durum.
`POST /v1/tasks/{id}/tool_approvals`	200 / 409	`{tool_call_id, decision:"approve"|"reject"}`. Durum requires_action değilse 409.
`GET /v1/conversations/{id}/messages?cursor=`	200	Geçmiş, cursor sayfalama.

SSE akışı (wire format)

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache
X-Task-Id: task_7d2

id: 1
event: task.started
data: {"task_id":"task_7d2","status":"running"}

id: 2
event: tool_call.started
data: {"tool_call_id":"tc_1","tool":"search",
       "args":{"query":"Q3 churn"}}

id: 3
event: tool_call.completed
data: {"tool_call_id":"tc_1","status":"ok","ms":640}

id: 4
event: message.delta
data: {"text":"Q3 churn was "}

: keep-alive   (15 sn'de bir yorum satırı, proxy
timeout'una karşı)

id: 57
event: task.completed
data: {"usage":
{"input_tokens":24180,"output_tokens":512}}
```

Neden SSE (WebSocket değil)?

- Akış tek yönlü (sunucu → istemci); girdi zaten ayrı POST'larla geliyor.
- Düz HTTP: LB, gateway, auth, HTTP/2 multiplexing ile sorunsuz.
- `id:` + `Last-Event-ID` ile **yerleşik devam** mekanizması.
- WebSocket: çift yönlü gerçek zaman (ses, ortak düzenleme) gerekirse.

Async mod (entegrasyonlar)

```
POST /v1/conversations/cv_1/tasks   {stream:false,
  "webhook_url":"https://customer.com/hooks"}
→ 202 { "id":"task_7d2","status":"queued" }
… sonra webhook:
POST https://customer.com/hooks
Example-Signature: t=1730880000,v1=hmac_sha256(...)
{ "type":"task.completed","task_id":"task_7d2", … }
```

Webhook: HMAC imza + timestamp (replay koruması), exp. backoff ile 24 saat retry, DLQ, müşteri için "redeliver" endpoint'i.

Hata ve kota cevapları

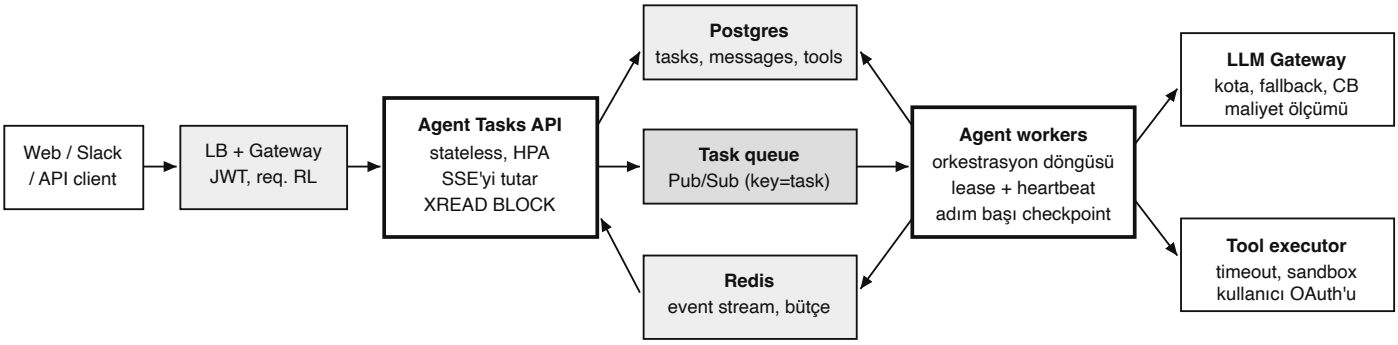
```
429 { "code":"TOKEN_BUDGET_EXCEEDED",
      "detail":"Tenant daily token budget used",
      "retry_after_s": 3600 }
409 { "code":"TASK_ALREADY_ACTIVE" }   // aynı
konuşmada 1 aktif task
422 { "code":"AGENT_TOOL_NOT_CONNECTED",
      "detail":"Link Jira first" }
```

Konuşma başına tek aktif task: `UNIQUE(conversation_id) WHERE status IN ('queued','running','requires_action')`. Mesaj sırası bozulmaz.

Veri modeli

```
tasks       (tenant_id, id PK, conversation_id, agent_id, status, idempotency_key, lease_owner, lease_until,
             checkpoint jsonb, input_tokens, output_tokens, error, created_at, finished_at)
messages   (tenant_id, id PK, conversation_id, task_id, role, content jsonb, created_at)
tool_calls (tenant_id, id PK, task_id, tool, args jsonb, status, result_ref, side_effect_key UNIQUE)
Redis:  stream  task:{id}:events (XADD, MAXLEN ~5000, TTL 24h)   ·   bucket  tokens:{tenant}
```

Senaryo B · Mimari ve derin dalışlar



Task başlat: API → Postgres INSERT (+outbox) → kuyruk → worker. Olaylar: worker → Redis XADD → API XREAD → SSE. Final mesaj ve kullanım Postgres'e.

1 · Stream ve devam etme

- Worker her olayı `XADD task:{id}:events` ile yazar (artan `seq`). API pod'u sadece okur → hangi pod'a bağlandığın önemsiz.
- Kopma: istemci `Last-Event-ID` ile tekrar bağlanır → `XRANGE` ile kaçırdıkları + `XREAD BLOCK` ile canlı.
- Redis stream'i 24 saat sonra silinir; kalıcı kayıt Postgres'te (final mesaj, tool çağrıları).
- **Deploy:** API pod'u SIGTERM alınca yeni bağlantı kabul etmez, mevcut stream'lere `retry: 1000` gönderip kapatır → istemci başka pod'a devam eder. LB idle timeout > keep-alive aralığı.

2 · Dayanıklı yürütme (worker crash)

- Worker task'ı **lease** ile alır (`lease_until`, heartbeat 10 sn). Lease biterse başka worker devralır.
- Her adımdan sonra `checkpoint` (mesaj geçmişi + tamamlanan tool sonuçları). Devralan worker oradan devam eder, baştan değil.
- Çok uzun/karmaşık iş akışları büyürse: Temporal gibi workflow motoru. Trade-off: yeni bir sistem işletmek.

3 · Tool yan etkileri: en fazla bir kez

- Yazan tool'larda `side_effect_key = task_id + tool_call_id` → hedef API'ye idempotency key olarak geçir veya önce DB'ye "started" yaz, sonuç gelince "done".
- Crash sonrası "started" ama "done" yok → hedefte kontrol et (ör. bu key ile ticket var mı?) veya kullanıcıya sor. **Körlemesine tekrar etme.**

4 · LLM Gateway ve kotalar

- Tenant başına **token bucket** (dakikalık hız + günlük bütçe), Redis Lua. Çağrı öncesi tahmini token rezerve, sonra gerçek kullanımla düzelt.
- Sağlayıcı 429/5xx: exp. backoff, sonra **fallback model/sağlayıcı**; circuit breaker. Timeout: ilk token için 10 sn, toplam için 120 sn.
- Öncelik: interaktif (web) > async (API) > batch. Yük altında batch'i kuyrukta beklet.
- Kullanım ve maliyet olayları → faturalama ve tenant dashboard'u.

5 · Güvenlik ve multi-tenancy

- Tool'lar **kullanıcının** OAuth token'ı ile çalışır (kullanıcının göremediğini agent de göremez).
- Prompt injection: tool çıktısı = güvenilmeyen veri; yazan tool'lar için insan onayı (requires_action), en az yetki scope'u.
- Task'lar ve olaylar tenant_id ile; Redis key'lerinde de tenant prefix.

SLO ve gözlemlenebilirlik

Task oluşturma availability	99.9%
Platform overhead (TTFT − model TTFT)	p95 < 500 ms
Task başarı oranı (kullanıcı hatası hariç)	> 99%
Kabul edilen task'ın kaybolması	0

Metrikler: kuyruk bekleme süresi, açık stream sayısı (pod başına), sağlayıcı bazlı hata/429, tokens/sn, tenant başına maliyet, tool hata oranı ve p95, lease devralma sayısı. **Trace:** task → her LLM çağrısı → her tool çağrısı bir span.

Kapanış cümlesi

🗣️ "*The main trade-off is the extra hop through Redis between worker and API. I accept it because it gives us resumable streams and lets us deploy either side without killing tasks.*"

Senaryolarda ince noktalar (ikinci göz)

Senaryo A ve B'yi çizdikten sonra bu listeyle kontrol et. Her satır, mülakatçının "what happens if…?" diye kurcalayacağı bir yer. Sol sütun tuzak, sağ sütun söyleyeceğin doğru cümle.

Senaryo A · Knowledge Sync

Tuzak	Doğru yaklaşım / cümle
202 = kalıcı kabul. Kuyruğa yazdım, DB'ye sonra yazarım.	Idempotency kaydı + sync satırı + outbox satırı **tek transaction**'da; commit'ten sonra 202. DB yoksa kabul etme. 🗣️ *I only return 202 once the work is durably recorded.*
Outbox relay bir kez yayınlar.	Relay de tekrar yayınlayabilir → consumer'lar tekrarı tolere eder (deterministik upsert / dedupe).
Tek bir yarış var.	İki ayrı yarış: **aynı istek** (unique idempotency kaydı, fingerprint = source + normalize body) ve **farklı istekler** (partial unique index ile tek aktif sync; mevcut sync'i 409/200 ile ancak **yetki kontrolünden sonra** göster).
Kaynak cursor'ı bir versiyon numarasıdır.	Opak bir ilerleme token'ı. Sadece ilgili iş kalıcı olduktan sonra ilerlet; token süresi dolarsa kontrollü yeniden tarama.
İçerik hash'i aynı → yapacak iş yok.	Embedding'i atla ama **izin değişikliği ve silme** yine işlenir. İçerik tazeliği ile izin tazeliğinin bütçesi farklı.
Ordering key = kilit.	Sıra korunur ama mesaj tekrar gelebilir; durum geçişlerini storage'da koru (version/generation koşullu update).
RLS açtım, izolasyon tamam.	Owner ve ayrıcalıklı roller RLS'i bypass eder. Uygulama rolüyle test et; pool'lu bağlantıda tenant'ı **transaction'a özel** ayarla (`SET LOCAL`); tenant-aware foreign key; negatif testler.
Token geçerli → doküman görülebilir.	JWT, claim'lerin güvenilir biri tarafından verildiğini kanıtlar; dokümanın **hâlâ** görünür olduğunu değil. Snippet ve LLM'e metin vermeden önce filtrele; cache key'e tenant + izin bağlamı girsin.
Tenant URL'de olmaz / token'daki tenant yeter.	URL'deki tenant yetkilendirilirse güvenli olabilir; token'daki tenant da **nesne bazlı kontrolü** ortadan kaldırmaz. Her istekte nesne + aksiyon kontrolü.
Hemen büyük dağıtık sistem.	1M doküman × günde %1 = 10.000/86.400 ≈ **0,12 değişiklik/sn**. v1: Postgres full-text; embedding'i ancak semantik arama gerekirse ekle (maliyet + silme/izinle senkron tutulacak bir index daha).

Senaryo B · Agent Tasks

Tuzak	Doğru yaklaşım / cümle
Lease yeterli.	Lease + **artan generation** (fencing token): eski worker'ın geç gelen yazımını storage reddeder.
Tool çağrısını "exactly once" yaparım.	🗣️ *I can't promise exactly-once external effects from our database alone.* Çağrıdan önce step ID + niyeti kaydet; hedef idempotency destekliyorsa aynı key'i kullan; desteklemiyorsa timeout sonrası sonuç belirsizdir → hedefte kontrol et ya da kullanıcıya sor.
Onay = "evet, devam".	Onay **tam aksiyona ve parametrelere** bağlı; parametre değişirse eski onay geçersiz.
Redis stream'den her zaman devam edilir.	Cursor sadece saklanan olaylarda çalışır. Süresi geçmişse belgelenmiş bir cevap (ör. 410) dön, istemci güncel task durumunu çeksin. Stream kalıcı geçmiş değildir.
Streaming = dayanıklı yürütme.	Streaming arayüzü iyileştirir, dayanıklılık sağlamaz. Kısa, sınırlı bir okuma için senkron cevap da makul olabilir.
Fallback model her zaman güvenli.	Fallback de aynı politika ve **veri lokasyonu** kurallarına uymalı. Süre, adım, tool çağrısı ve token için bütçe; pahalı adımdan önce rezerve et.
Her task için metrik.	Task ID'leri metrik etiketi yapma (cardinality). Task detayı trace'te; metrikler: kuyruk gecikmesi, tamamlanma oranı, adım hatası, ilk faydalı çıktıya kadar süre, maliyet.

Referanslar: AWS transactional outbox, PostgreSQL row security, Google Cloud Pub/Sub message ordering. Senaryolar alıştırma tasarımıdır, AI Company'nin gerçek mimarisi değildir.

Kod yazdırırlarsa: Go endpoint iskeleti

Dil seçimi: En hızlı olduğun dili seç (burada Go). Başta sor: 🗣️ *"Is it OK if I write this in Go? It's where I'm fastest, and I'm comfortable reading TypeScript."* Sıra: **validate → tenant (token'dan) → idempotency → iş → doğru status → tek hata formatı**. (Bu kod derlendi ve 7 senaryoyla test edildi.)

```
package main
import (...) // context, crypto/sha256, encoding/hex,
  // encoding/json, errors, log/slog, net/http, time

type SyncRun struct {
  ID          string    `json:"id"`
  TenantID    string    `json:"-"`
  SourceID    string    `json:"source_id"`
  Mode        string    `json:"mode"`
  Status      string    `json:"status"`
  CreatedAt   time.Time `json:"created_at"`
}

// one error shape for the whole API
type APIError struct {
  Status int    `json:"status"`
  Code   string `json:"code"`
  Title  string `json:"title"`
}

func (e *APIError) Error() string { return e.Code }

// in-memory fake in tests, Postgres in prod
type Store interface {
  SourceExists(tenant, source string) bool
  ByIdempotencyKey(tenant, key string) (SyncRun, string, bool)
  ActiveRun(tenant, source string) (SyncRun, bool)
  // prod: UNIQUE(tenant_id, key) + outbox row, one txn
  Create(run SyncRun, key, bodyHash string) error
}

type Server struct {
  store Store
  newID func() string
  now   func() time.Time
}

func (s *Server) Routes() *http.ServeMux {
  mux := http.NewServeMux() // Go 1.22 patterns
  mux.HandleFunc("POST /v1/sources/{id}/syncs",
s.handle(s.createSync))
  return mux
}

type handlerFunc func(w http.ResponseWriter, r *http.Request)
error

func (s *Server) handle(h handlerFunc) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    if err := h(w, r); err != nil {
      var ae *APIError
      if !errors.As(err, &ae) {
        slog.Error("unhandled", "err", err,
          "request_id", r.Header.Get("X-Request-Id"))
        ae = &APIError{500, "INTERNAL", "unexpected error"}
      }
      writeJSON(w, ae.Status, ae, "application/problem+json")
    }
  }
}
```

```
func (s *Server) createSync(w http.ResponseWriter, r
*http.Request) error {
  // set by auth middleware from the verified JWT
  tenant := TenantFrom(r.Context())
  source := r.PathValue("id")
  key := r.Header.Get("Idempotency-Key")
  if key == "" {
    return &APIError{400, "IDEMPOTENCY_KEY_REQUIRED", "missing
key"}
  }
  var body struct {
    Mode string `json:"mode"`
  }
  r.Body = http.MaxBytesReader(w, r.Body, 1<<20)
  if err := json.NewDecoder(r.Body).Decode(&body); err != nil
{
    return &APIError{400, "INVALID_JSON", "bad json"}
  }
  if body.Mode == "" {
    body.Mode = "incremental"
  }
  if body.Mode != "full" && body.Mode != "incremental" {
    return &APIError{400, "INVALID_MODE", "full|incremental"}
  }
  if !s.store.SourceExists(tenant, source) {
    // same 404 for another tenant's id: no leak
    return &APIError{404, "NOT_FOUND", "source not found"}
  }
  sum := sha256.Sum256([]byte(body.Mode))
  hash := hex.EncodeToString(sum[:])

  if prev, h, ok := s.store.ByIdempotencyKey(tenant, key); ok
{
    if h != hash {
      return &APIError{409, "IDEMPOTENCY_KEY_REUSED", "body
differs"}
    }
    w.Header().Set("Location", "/v1/syncs/"+prev.ID)
    writeJSON(w, 202, prev, "") // replay first answer
    return nil
  }
  if active, ok := s.store.ActiveRun(tenant, source); ok {
    w.Header().Set("Location", "/v1/syncs/"+active.ID)
    writeJSON(w, 200, active, "") // documented choice
    return nil
  }
  run := SyncRun{ID: "syn_" + s.newID(), TenantID: tenant,
    SourceID: source, Mode: body.Mode,
    Status: "queued", CreatedAt: s.now().UTC()}
  if err := s.store.Create(run, key, hash); err != nil {
    return err // real store: unique violation → replay/409
  }
  w.Header().Set("Location", "/v1/syncs/"+run.ID)
  writeJSON(w, 202, run, "")
  return nil
}

func writeJSON(w http.ResponseWriter, status int, v any, ctype
string) {
  if ctype == "" {
    ctype = "application/json"
  }
  w.Header().Set("Content-Type", ctype)
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(v)
}

type tenantKey struct{}

func TenantFrom(ctx context.Context) string {
  t, _ := ctx.Value(tenantKey{}).(string)
  return t
}
```

Yazarken sesli söyle

- "The tenant comes from the verified token, never from the request body."
- "Storage is behind an interface, so I can test with an in-memory fake and swap in Postgres."
- "In production the race between two identical requests is closed by a unique constraint, not by this if-check."
- "Insert and publish can diverge if we crash in between, so in production I'd use a transactional outbox."

Test senaryoları (göster!)

- Aynı key + aynı body → aynı run, 202. Aynı key + farklı body → 409.
- Key yok → 400. Geçersiz mode → 400. Başka tenant'ın kaynağı → 404.
- Aktif run varken yeni key → mevcut run, 200.
- `httptest.NewRecorder()` + context'e tenant koyarak handler testi.

Mini drill'ler: 10'ar dakikada kağıda çiz

Her biri için: 3 netleştirme sorusu → 4-6 endpoint → 1 tablo → 1 risk. Sonra ipuçlarıyla karşılaştır. (Yolculukta en verimli pratik bu.)

Problem	Kontrol listesi / ipuçları
1. Internal Deployment API (DX platformu: takımlar servislerini deploy eder). *İnfra rolüne en yakın soru!*	`POST /v1/services/{id}/deployments {image_digest, env, strategy:"canary"}` → 202 · `GET …/deployments/{id}` (pending → progressing → succeeded | failed | rolled_back) · `POST …:rollback` · `POST …:promote`. Tag değil **digest** (değişmez). Ortam başına aynı anda 1 deploy (lock). Canary: %5 → metrik analizi (hata oranı, p95) → otomatik rollback. Audit log, kim onayladı. Arkada ArgoCD/GitOps: API git'e commit eder ya da Argo API'sini çağırır.
2. Webhook delivery sistemi (dışarıya olay gönderme)	Abonelik CRUD; olay → outbox → kuyruk → gönderici. HMAC imza + timestamp. Retry: exp. backoff 24 saate kadar, sonra DLQ + endpoint'i devre dışı bırak + e-posta. Endpoint başına circuit breaker ve eşzamanlılık limiti (yavaş müşteri diğerlerini bloklamaz). Sıralama garanti etme, `event_id` + `created_at` ver. `POST /v1/events/{id}:redeliver`.
3. Public API: API key + rate limiting	Key üretimi: sadece bir kez göster, DB'de hash (SHA-256), prefix ile arama. Scope'lar, son kullanım zamanı, rotasyon (iki key aynı anda geçerli). Token bucket Redis Lua; boyutlar: key, tenant, endpoint. Header'lar: `RateLimit-*`, `Retry-After`. Redis çökerse: lokal limit + fail-open (okuma) / fail-closed (pahalı LLM).
4. Büyük dosya yükleme (öğrenme ürünü: video, PDF)	`POST /v1/uploads {filename,size,content_type}` → GCS **resumable signed URL** → istemci direkt yükler → GCS event → işleme (virüs taraması, transcode, metin çıkarma) → `GET /v1/uploads/{id}` durum. Boyut/tip limiti, checksum (MD5/CRC32C), süresi dolan yarım yüklemeleri lifecycle ile sil.
5. Kurs kaydı ve ilerleme (öğrenme ürünü)	`PUT /v1/courses/{c}/enrollments/{user}` (idempotent). İlerleme: `POST …/progress-events {lesson_id, event:"completed", client_ts, event_id}`: offline mobil → event_id ile dedupe, sırasız gelirse max(…). Admin raporları ağır → read replica / analytics (BigQuery) export. Tamamlama sertifikası async.
6. Audit log API (kurumsal müşteri isteği)	Append-only; `GET /v1/audit-events?actor=&action=&from=&to=&cursor=`. Zamana göre partition, tenant'a göre filtre, retention tenant ayarı. Değiştirilemezlik: hash zinciri veya WORM bucket. SIEM'e export (webhook / GCS günlük dosya). Yazma yolu ana isteği yavaşlatmasın: outbox → async.
7. SCIM kullanıcı provizyonu (Okta/Entra → AI Company)	SCIM 2.0 standardı: `/scim/v2/Users`, `/Groups`, PATCH ops. `externalId` ile eşleştir. Deaktive → oturumları ve refresh token'ları iptal et, ACL cache'i temizle. Toplu işlemlerde kısmi hata raporu. Tenant başına bearer token.
8. Feature flag / config servisi	Okuma ağırlıklı: SDK lokal cache + streaming güncelleme (SSE) + periyodik poll yedeği. Değerlendirme lokal (ağ çağrısı yok, lokal JWKS doğrulaması gibi). Kill switch. Değişiklik audit'i, yüzde bazlı rollout (deterministik hash(user_id)). Servis çökerse son bilinen config.

Kendi çizimin için boş alan

Vaka çalışması · Bir auth platformunun evrimi

Çok kiracılı, çok sayıda micro-frontend'den oluşan kurumsal bir platformda kimlik doğrulama ve yetkilendirmenin adım adım evrimi. Her adımda gerçek alternatifler, kabul edilen trade-off'lar ve sonradan ortaya çıkan sınırlar var. İzin farkındalıklı API tasarımında (Senaryo A) doğrudan işe yarar.

Adım	Problem	Karar	Kabul edilen trade-off
0 · Önce	JWT'yi şişirmemek ve kullanıcı bilgisi çağrısını tek yerde toplamak için herkesin önünde merkezi bir session servisi.	Paylaşılan session ID, 4 saatlik kayan süre; her BFF isteği session'ı doğrular.	Her istekte ekstra hop, SPOF; sızan session ID'yi durdurmak elle müdahale ister.
1 · Token AuthN	Güvenlik (sızan session), gecikme, her MFE'nin kendi refresh mantığı.	Kısa ömürlü access token (~5 dk) + **HTTP-only cookie'de refresh token**; refresh shell'de merkezi; public key ile **lokal doğrulama**; minimal payload, PII yok.	İptal, access token ömrü kadar (≤5 dk) gecikebilir.
2 · İzin modeli	Sadece rol: kaba taneli, her uygulama kendi eşlemesini tutuyor, **denetlenemiyor**.	Bounded context başına **scope**; rol ile uygulama izni arasında **permission group**; market/bayi/kullanıcı tipi **filtreleri**; denetim için **permission snapshot**.	Karmaşıklık, yönetişim yükü, token boyutu.
3 · OIDC akışı	Bu token'ı kim, nasıl üretecek? Standart OIDC, hassas veri yok, bakımı kolay.	5 alternatif PoC → **Ory (Kratos + Hydra)**. `verify` / `getContexts` kütüphaneleri arka tarafı soyutlar.	Self-hosting ve işletme bizde.
4 · Token exchange	Merkezi güvenlik ekibi, hangi uygulamanın hangi token'ı kullandığını Entra'da göremiyordu.	**RFC 8693 Token Exchange**: Entra access token → kurum token'ı. Format ve kütüphaneler aynı kaldı.	Ekstra istek; imzalama kodu ekstra inceleme ister.
5 · Sonra	Token boyut tavanı: çok sayıda pazara yetkili kullanıcılar token'a sığmıyor; statik rol eşlemesi; marka bazlı ve US/EU ayrımı ihtiyacı.	Merkezi karar servisi: `POST /authz/check`, ABAC, sandbox'lı JS policy'leri, cache, append-only audit.	Kritik yolda ağ çağrısı (<20 ms p95 hedefi); ekiplerin yazdığı kodu çalıştırma riski.

Neden Ory? (alternatifler)

Entra + custom UserInfo	En basit, ama her istekte ekstra çağrı ve gecikme; sadece Entra; offline yok.
Entra custom claims	Değişiklik için Global Admin; düz claim yapısı, JSON yok; loglar ekipte değil.
Keycloak	Kullanıcıların ayrı kopyası + senkronizasyon; özelleştirme Java'da; dik öğrenme eğrisi.
Ory ✓	Standart OIDC/OAuth2, en esnek token hook, çoklu IdP, olası offline, iyi admin araçları. Bedeli: self-hosted.

Hazır cevaplar (İngilizce)

Why Ory? 🗣️ *We needed standard OIDC so any team could use off-the-shelf libraries, a token hook to add our permission claims, and room for more than one identity provider. Entra alone meant an extra user-info call per request, or admin-only custom claims with a flat structure. Keycloak meant syncing a copy of all users and writing Java plugins. Ory gave us all of it; the price was running it ourselves.*

Biggest trade-off? 🗣️ *Putting authorization data in the token made checks local and fast, but revocation was slower and the token kept growing. Later the team hit a size ceiling for users with many markets and moved fine-grained authorization to a central, cached decision service.*

How did you make adoption easy? 🗣️ *We wrapped verification in small libraries, so teams called verify or getContexts and never parsed tokens themselves. That also let us change the backend later without breaking anyone.*

Security team? 🗣️ *After it was live, the central security team wanted visibility in Entra. We kept the token format and libraries and switched to standard OAuth token exchange, so consumers barely noticed.*

Do differently? (önerilen yansıma, katılıyorsan kullan) 🗣️ *I'd keep authentication in the token but plan for fine-grained authorization as a cached decision API earlier, because the token-size limit was predictable.*

AI Company'ye köprü: Senaryo A'daki izin farkındalıklı arama aynı soru: izinler token'da mı, indekste mi, merkezi bir kontrolde mi? Bu sayfadaki trade-off'ları orada kullan.

25 dakikalık kağıt mock + hafıza düzeltmeleri

Mock akışı (zamanlayıcı kur)

Dakika	Yap
0-3	Knowledge Sync veya Agent Tasks seç. 3 netleştirme sorusu yaz.
3-8	4 endpoint + 1 tam request/response yaz.
8-13	Tablolar, tenant sınırları, durumlar.
13-20	Bir crash, bir retry ve bir izin iptalini baştan sona izle.
20-25	Ana trade-off'u ve bir operasyonel kontrolü **sesli** söyle.

Kendini puanla

Kontrol	İyi bir cevap neyi gösterir	✓
Sözleşme	İstemci ne olduğunu ve sonra ne yapacağını anlıyor.	
Doğruluk	Tekrarlanan istek iki kez iş başlatmıyor; çakışan isteklerin sonucu tanımlı.	
Güvenlik	Geçerli kimlikten sonra güncel nesne + aksiyon kontrolü var.	
Kurtarma	Crash sonrası kalıcı durumu bulabiliyorsun; belirsiz yan etkileri açıklayabiliyorsun.	
Operasyon	Hatayı fark etme, güvenli deploy, geri dönüş.	
İletişim	Önce soruya cevap, faydalı detaydan sonra dur.	
Sadelik	5 dakikada çizilebilen bir v1 vardı; karmaşıklık sorulunca geldi.	

Takıldığında reset cümlesi

🗣️ *Let me trace one request and find the first place where state changes.*

Hafızanı düzelt: 8 yanlış refleks

1. Kuyruk sıralaması = sahiplik/kilit **değil**.
2. JWT doğrulaması = anında iptal **değil** (kısa ömürlü token: iptal, token süresi kadar gecikebilir).
3. İçerik değişmedi = izin değişmedi **değil**.
4. 202 = kalıcı kabul. Kaydetmediğin işi kabul etme.
5. Timeout sonrası dış yan etki **belirsiz** olabilir.
6. Mülakat aracında bir özelliğin görünmesi = kullanma izni **değil**. Sor.
7. Mutex sadece tek process'i korur; birden çok replika = DB transaction / atomik update.
8. Davette süre yazmıyor: 60 dk bir **prova varsayımı**, başta sor.

Notların

Mock için çizim alanı